CHAPTER

5

RESEARCH ARCHITECTURE AND TRUST-CENTRIC CONSENT MODEL

5.1 Introduction

This chapter first presents the research architecture on which the framework is built. There are three layers in the architecture, and we explain the role of each layer. We then propose a model that represents patient consent as an evaluable authorisation object; it addresses research objective 1 at the conceptual and prototype level, and Section 5.9 gives the evidence. In this chapter, we also present the results of the validation of the proposed solution to research question one. To validate this research question, we built the HALAH prototype and used the following technologies to test the system:

1. **Blockchain:** A blockchain is a decentralised, distributed ledger: an entry, once recorded, cannot be altered without altering every block after it (Wood 2014). In HALAH the chain holds no medical content; it anchors only the state of an approved consent, as a soulbound token (Cai 2022) that cannot be transferred, and its revocation. The Ethereum test network Sepolia (Ethereum Foundation 2026b) was chosen because it runs the same virtual machine as the main network, so the contract executes as it would on the main network, while its ether has no monetary value. The contract was deployed through the MetaMask wallet (Consensys 2026), and every transaction is publicly visible on Etherscan.
2. **Solidity:** The consent contract ConsentSBTv3 is written in Solidity, the contract language of Ethereum (Ethereum Foundation 2026a), on the OpenZeppelin implementation of the ERC-721 token standard (Entriken et al. 2018) with the ERC-5484 soulbound interface (Cai 2022), and was compiled in the Remix environment (Ethereum Foundation 2026c). The contract stores the start time, the expiry time and the revocation flag of each consent and reports whether the consent is valid at a given time, so the temporal and lifecycle conditions of Req 5 can be checked against the chain by any party, independently of the application.
3. **Python and Streamlit:** The Controller and the four role interfaces of the prototype, for the patient, the guardian, the doctor and the auditor, are written in Python with the Streamlit library, and the application reaches Sepolia through the web3 library. Consents, approvals, decisions and the audit ledger are kept in a local SQLite

database, so the auditor can query the evidence of every decision while nothing but the consent state goes to the chain.

4. **Decentralised identifiers and verifiable credentials:** Every participant is named by a decentralised identifier (W3C 2022), and the doctor and each guardian hold a role credential (W3C 2025) signed by a trusted issuer, so participant identity and authority are treated as separate conditions, as Chapter 6 presents.
5. **Synthea:** The prototype uses synthetic patient records produced by the Synthea generator (Walonoski et al. 2018). The dataset is described in Section 5.7.

The outline of this chapter is as follows. In Section 5.2, we outline the three layers of the architecture and their components. In Section 5.3 we explain where trust is evaluated and where the boundaries between the layers lie, and in Section 5.4 we map the architecture to the six research objectives. In Section 5.5, we present the trust-centric consent model: the creation workflow, the structure of the consent object C = (I, A, P, T, L, E), and the creation procedure as Algorithm 1. In Section 5.6, we describe the aims of the prototype used to validate the proposed consent model, and in Section 5.7 the dataset used for validation. Section 5.8 explains step by step how the prototype creates consent. Finally, in Section 5.9 we discuss the results obtained from the validation of the consent model and the evaluation of the architecture, and Section 5.10 concludes the chapter.

5.2 Layered Architecture and Component Decomposition

The proposed research architecture adopts a Model–View–Controller (MVC) design to structure the implementation of the trusted multi-party consent management framework. In the proposed framework, MVC does more than separate data, interfaces, and application logic. It also keeps three things apart: the consent itself, the independent stakeholders who act on it, and the place where the authorisation decision is enforced. A consent decision in healthcare can involve patients, guardians, healthcare professionals and auditors, each of whom may hold a different identity and authority. The system therefore cannot decide access from a single authentication event or a static permission entry. Instead, it checks several conditions together, each verifiable on its own; Section 5.3 sets them out.

At the highest level, the architecture consists of three complementary layers. The View Layer represents the interaction boundary with independent healthcare stakeholders. The Controller Layer represents the security decision and coordination boundary. The Model Layer represents the persistent trust, consent, policy, and authorisation state. Figure 5.1 shows the three layers and how they relate. The healthcare data sit outside the three MVC layers; the View never reaches them directly, and every access passes through the Controller.

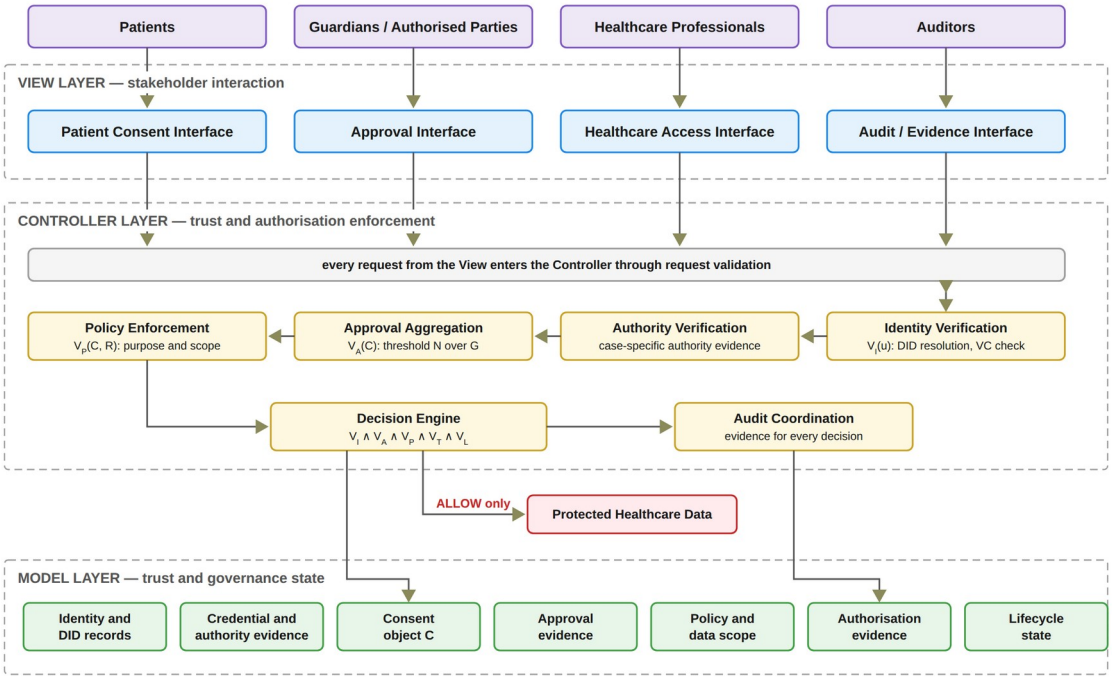


Figure 5.1: Layered MVC architecture and component decomposition of the HALAH framework.

What a user submits through the View is a request or a claim; it is not an authorisation decision. The Controller Layer is the principal security enforcement boundary. It receives requests from the View and coordinates the evaluation of the relevant trust conditions. These include decentralised identity verification, credential and authority verification, multi-party approval evaluation, policy and data-scope evaluation, and final authorisation. The Controller also generates audit evidence associated with the resulting decision. The Model provides the information required for the Controller to evaluate an access request, but the existence of a Model object does not itself imply that access is permitted. MVC was selected in preference to a purely on-chain design for two reasons. First, the authorisation decision must combine the identity, threshold, policy, temporal, and lifecycle evaluation performed by the Controller with the anchoring of consent state on the blockchain. Second, each stakeholder interface can be replaced without altering the enforcement logic, so the trust predicates are evaluated independently of any particular user interface.

Figure 5.2 presents this decomposition. The View is decomposed according to the principal stakeholder roles. Patients specify consent and perform consent-related actions through the patient interface. The guardian or other authorised-participant interface supports independent approval. Healthcare professionals use their interface to submit authorised data-access requests. The auditor interface supports inspection of authorisation evidence and audit history. These interfaces have different responsibilities, but none independently determines whether access should be granted. The Controller is decomposed into six principal security functions. Identity Verification establishes whether a participant can be identified through the relevant

decentralised identity mechanism. Authority Verification checks whether the participant has appropriate evidence of the required role or authority. Approval Aggregation determines whether independently verified approvals satisfy the predefined threshold. Policy Enforcement evaluates whether the requested purpose and data scope comply with the consent policy. Decision Engine combines the resulting predicates into the final authorisation decision. Finally, Audit Coordination records the relevant evidence and decision events. The Model is decomposed into seven principal state representations: DID, VC, Consent, Approval, Policy, Evidence, and Lifecycle. This decomposition establishes an explicit distinction between state, interaction, and enforcement.

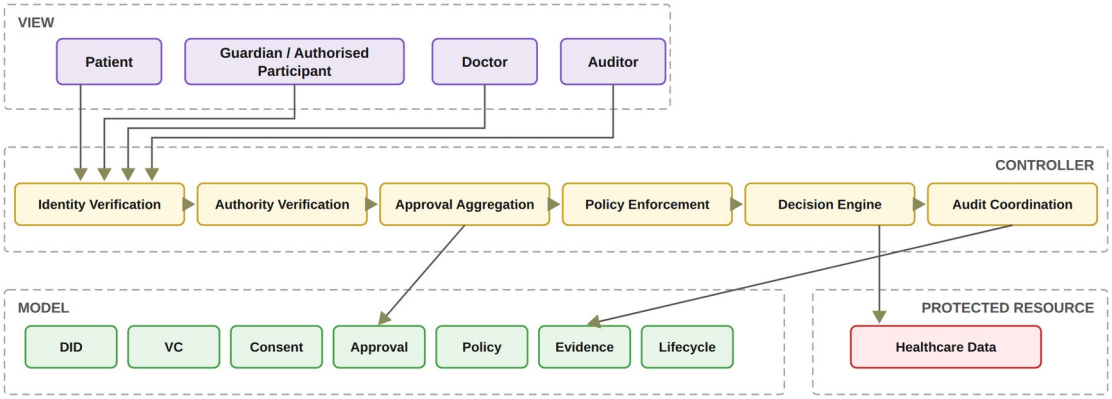


Figure 5.2: Functional decomposition of the MVC components.

In the implemented prototype, the Controller comprises eight principal functions rather than six, since a Validity Proof Service checks the consent once more before any data are released, as Chapter 8 presents, and an On-chain Anchor Service anchors the Sepolia mint as a background process. A revoke transaction is sent when the patient revokes a consent, and the patient view waits for its confirmation. The Model correspondingly contains the ConsentSBTv3 soulbound contract deployed on Sepolia together with the application audit ledger, both of which an independent verification tool reads, as described in Chapter 10. In the present prototype, each DID must be well formed, and the doctor and each guardian must hold a valid verifiable credential from a demo issuer. The guardian's credential is checked before the guardian signs, and V_I combines the doctor's credential check with the EIP-712 verification of each listed guardian's signature. The requester's DID is declared in the interface rather than authenticated, and wallet-held signing keys, full DID resolution and proof that the user controls the DID are deferred to the next stage. However, each guardian approval is an EIP-712 typed-data signature bound to the consent identifier, so the multi-party threshold is enforced over signed approvals rather than over interface actions.

5.3 Trust Evaluation and Architectural Boundaries

The three layers do not call each other freely; a request moves through them in a fixed order. As illustrated in Figure 5.3, the Controller coordinates a sequence of trust predicates representing the conditions established by the sub-research questions. The first condition, $V_I(u)$, determines whether the requesting participant possesses valid identity and authority evidence. The second condition, $V_A(C)$, determines whether the required independently verified participants have collectively satisfied the authorisation threshold N. The third condition, $V_P(C, R)$, determines whether the requested operation complies with the consent policy, including purpose and data-scope restrictions. The fourth condition, $V_T(C, t)$, determines whether the consent remains valid at the time of the request. The fifth condition, $V_L(C)$, determines whether the consent remains in a valid lifecycle state. The integrated decision can therefore be expressed as Equation (5.1). In the current prototype the Consent Engine derives the lifecycle state, and a request goes forward only when that state is ACTIVE; the revocation flag and the token are checked within V_A as well.

Equation (5.1) states that a request is allowed only when every one of the five conditions holds at the time of the request. Conversely, the failure of any one condition is sufficient for refusal, as Equation (5.2) states, and the success of the other conditions cannot compensate for it.

$$\text{Decision}(C, R, t) = \text{ALLOW} \iff V_I(u) \land V_A(C) \land V_P(C, R) \land V_T(C, t) \land V_L(C). \tag{5.1}$$

$$\neg(V_I \land V_A \land V_P \land V_T \land V_L) \Rightarrow \text{Decision}(C, R, t) = \text{DENY}. \tag{5.2}$$

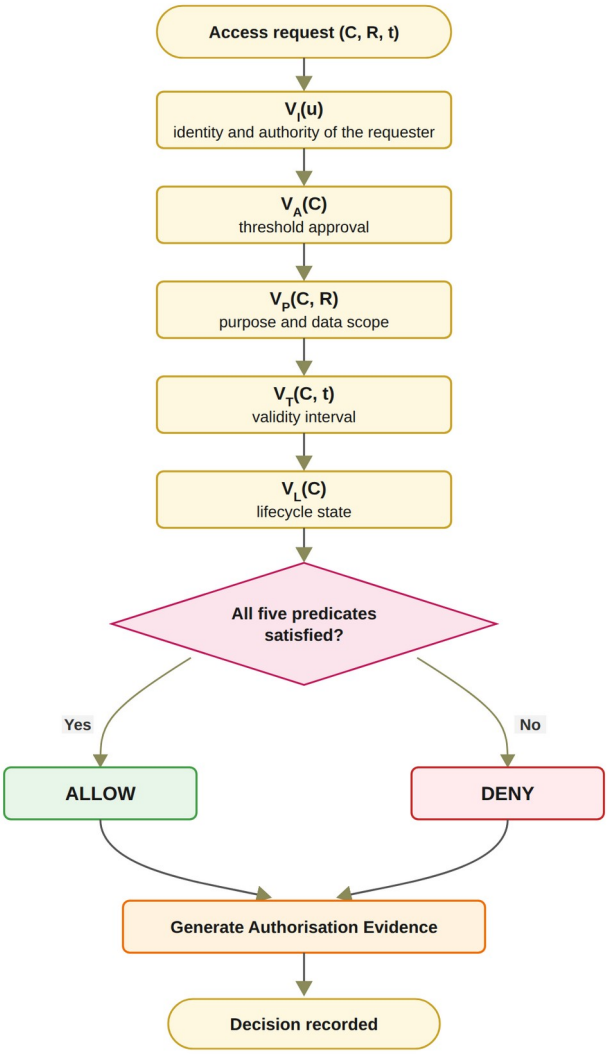


Figure 5.3: Trust predicate evaluation and authorisation decision flow.

The Controller does not itself create these trust conditions. Instead, it evaluates evidence represented by the Model and coordinates the resulting decision. In short, the Controller decides and enforces; the Model holds the trust and governance state. Every decision, allowed or denied, is recorded in the audit ledger together with its authorisation evidence, so that it can be examined later. Each layer also marks a trust boundary. The View is the interaction boundary, and, as Section 5.2 noted, what it submits is never a decision. The Controller is the security enforcement boundary. It applies the same rules to every request and releases healthcare data only when all five predicates hold. The Model is the security-state boundary. Its records provide the evidence and state required for authorisation evaluation, but stored state alone does not constitute permission. The protected healthcare data represent the final resource boundary, and access is permitted only through the authorisation path enforced by the Controller. The proposed framework does not claim that MVC itself provides security. Instead, security arises from the explicit placement of trust-sensitive operations within the Controller, the structured representation of security state within the Model,

and the prevention of direct security decisions at the View layer. In the present prototype, the Controller is a single trusted server that both evaluates the predicates and writes the audit ledger. Only the consent state is anchored on Sepolia, so the independent verification tool can check the Controller's consent records against the chain, while the remaining ledger entries rest on trust in the Controller. Figure 5.4 puts the same conditions in the order a request meets them. A DID identifies who claims to be asking, a verifiable credential and a signed approval answer what that participant may do, the threshold answers whether the members of G agreed, and the policy, temporal and lifecycle checks answer whether the consent is still valid for this request. The figure also states, gate by gate, what the current prototype checks, so the reader can see where the design and the prototype differ before the detailed sections.

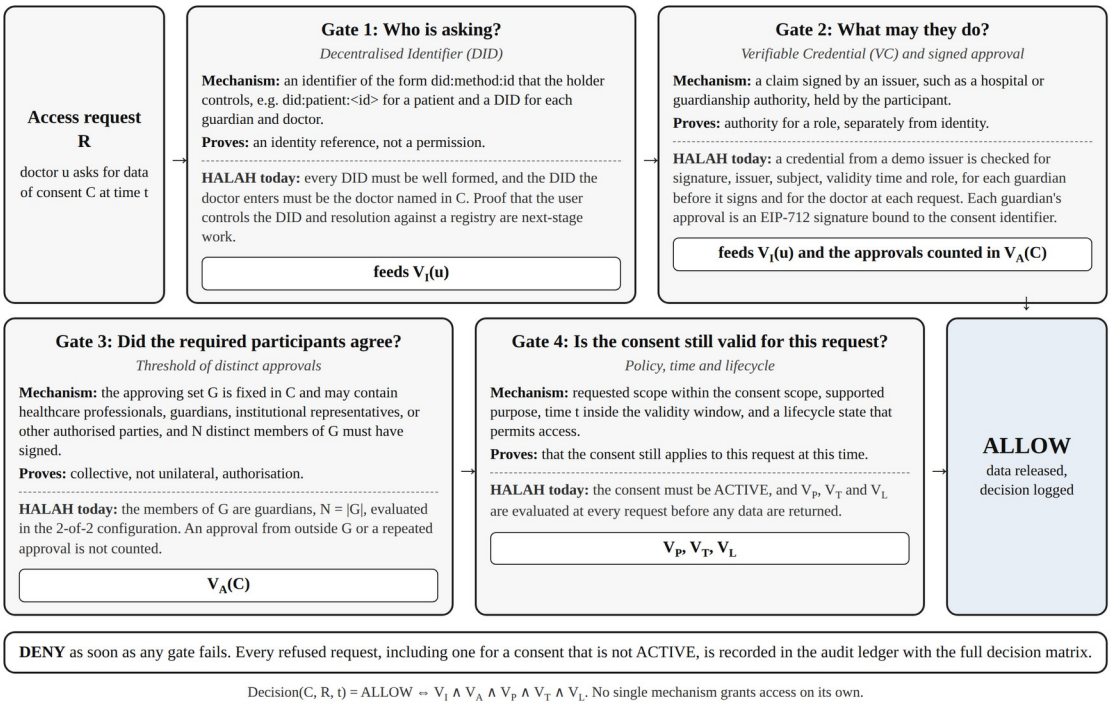


Figure 5.4: The four gates that an access request passes before data are released.

5.4 MVC Architecture and the Research Objectives

The MVC architecture also provides a direct implementation structure for the research objectives, as summarised in Table 5.1. RO1 is primarily represented in the Model through the development of the trust-aware consent object C = (I, A, P, T, L, E). RO2 is implemented through the identity and authority verification mechanisms coordinated by the Controller. RO3 is realised by the Controller's threshold check over the verified approvals. RO4 is implemented through the policy evaluation mechanisms that constrain the authorised purpose and healthcare data scope. For RO5, temporal conditions and lifecycle state are represented in the Model, while V_T and V_L are evaluated by the Controller before an access decision is produced. RO6 operates across the complete architecture rather than being confined to a single MVC layer. Formal assurance of the integrated model's logical

properties is left to the next stage; the current work tests empirically whether the prototype enforces them. The evidence generated by the Controller and stored in the Model is what the empirical validation and the independent verifier examine.

Table 5.1: Mapping between research objectives and MVC components

Research Objective	Primary MVC Component	Core Mechanism	Expected Contribution
RO1	Model	Trust-aware consent representation	Consent as the evaluable object C = (I, A, P, T, L, E) rather than a stored permission.
RO2	Controller and Model	DID and VC verification	Identity (DID) checked separately from role authority (VC); proof of DID control deferred to the next stage.
RO3	Controller	Verified multi-party threshold evaluation	Every guardian in G must sign (N = |G|); with two or more guardians, none can approve alone.
RO4	Controller and Model	Policy, purpose, and data-scope evaluation	Access bounded by the consent's purpose and data scope, whatever the requester's identity and approvals; the interface currently fixes the purpose to Treatment.
RO5	Model and Controller	Temporal validity and consent lifecycle enforcement	Validity window and lifecycle state checked at every request; access only while ACTIVE, refused once expired or revoked.
RO6	Model, View, and Controller	Formal security analysis and empirical validation	Empirical validation of the decisions the prototype allows and denies (Chapter 10); formal assurance of the model left to the next stage.

Table 5.2 summarises how the proposed framework differs from traditional EHR access control.

Table 5.2: Traditional EHR access control compared with the HALAH framework

Aspect	Traditional EHR access control	HALAH framework
Identity	Central identity provider; institution login	A DID for every participant; identity kept separate from authority
Consent	A stored record that permission was given	An object C = (I, A, P, T, L, E), checked at every request
Approval	A static permission set by one party	All listed guardians sign (EIP-712); with two or more guardians, no single party can approve alone
Access check	Log in, then check a static permission	VI ∧ VA ∧ VP ∧ VT ∧ VL, evaluated at every request
Time and revocation	Hard to verify once the permission is stored	Validity window and revocation checked each time; on-chain revoke
Audit	A record that consent occurred	Every access decision, allowed or denied, logged off-chain with its decision matrix; consent state anchored on Sepolia

The contribution of the proposed architecture is the way MVC is used to carry a trust-oriented authorisation model, not the use of MVC itself. Under this design, multi-party consent is managed by checking its conditions at each access rather than by storing a record of permission. The evaluable consent object as such is already present in the literature. What is new is that a consent takes effect only when identity, authority, collective approval, policy, time and lifecycle all hold together, and that these conditions are checked on every access decision (Equation (5.1)). Blockchain or other tamper-resistant storage mechanisms are integrated as infrastructure for preserving selected authorisation evidence. However, the blockchain itself is not treated as the source of trust. In HALAH, it holds only the approved consent state and its revocation, so the consent state in the Controller's records can be checked against it. The detailed solutions for RQ1 to RQ6 are presented in Chapters 5 to 10, and the formal assurance of the complete decision model remains for the next stage. The next section presents the solution for RO1, the consent object on which the other objectives build.

5.5 Trust-Centric Consent Model

The consent object below is our solution to RO1. Section 5.5.1 shows the consent creation workflow, Section 5.5.2 defines the structure of the consent object, and Section 5.5.3 presents the creation procedure and the algorithm; the prototype, dataset and validation follow in Sections 5.6 to 5.9.

RO1 addresses RQ1 by developing a trust-aware consent model in which patient consent is a structured and evaluable authorisation object. The aim is

to formally represent what the patient intends to authorise, who is permitted to participate in the authorisation process, which healthcare data may be accessed and under what conditions. The proposed consent object is represented as Equation (5.3).

$$C = (I, A, P, T, L, E) \tag{5.3}$$

In Equation (5.3), I represents patient and participant identity information, A represents authorisation requirements, P represents policy constraints, and E represents authorisation evidence. T represents the temporal validity window, and L represents the current state of the consent object and its permissible transitions. The identity component holds the DIDs of the patient, the doctor and the guardians. The authorisation component defines the required participants and approval threshold. The policy component specifies the permitted purpose and healthcare data scope. The evidence component records the information required to support subsequent verification and audit. A consent object does not authorise requests by itself. It holds the conditions against which each request is checked.

5.5.1 Consent Creation Workflow

Figure 5.5 describes the creation of a trust-aware consent object. The process starts with the patient's request and ends with a stored consent record that the later checks can evaluate. The workflow first collects the patient identity and the intended healthcare participants. It then specifies the purpose and data scope of the consent. In the HALAH implementation, the patient interface allows the patient to select a patient DID, specify an authorised doctor, identify guardian DIDs, and select healthcare data modules such as Observation, Medication, Condition, and Procedure. The implementation also collects the start and expiry time of the consent. In the current interface, the purpose is fixed to Treatment and is not selected by the patient, and the purpose vocabulary check is therefore exercised by the automated tests rather than by the interface.

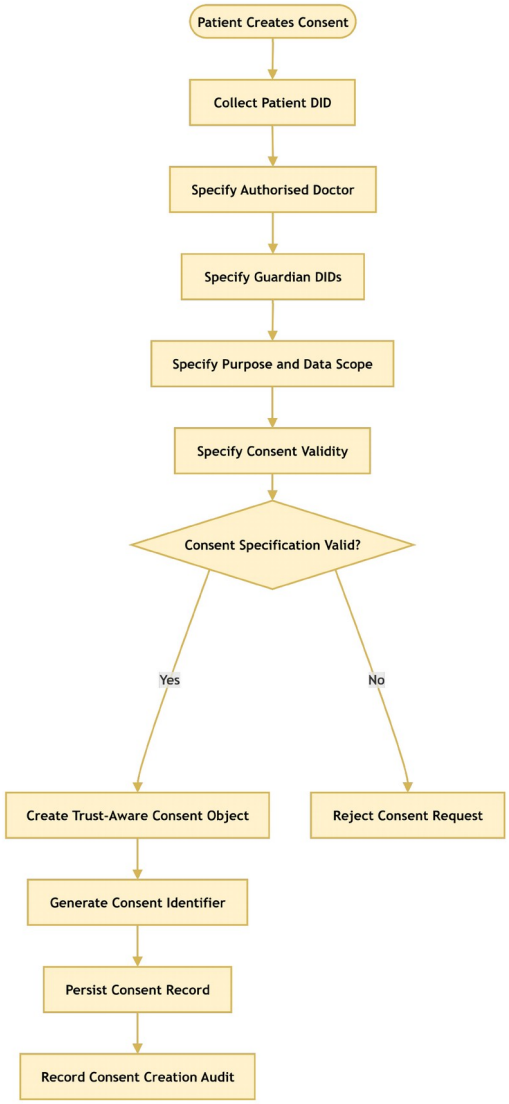


Figure 5.5: RO1 workflow for trust-centric consent creation.

The specification is accepted only when the required fields are valid. For example, the expiry time must be later than the start time, the data scope must not be empty, and each guardian DID must be well formed. After validation, the system creates the consent object, gives it an identifier, stores it, and writes the creation event to the audit layer. Once the consent object is created, its lifecycle state L becomes PENDING, because the consent cannot produce an effective authorisation decision until the required approvals have been obtained. In the current prototype, this state is PENDING_SIGNATURE.

5.5.2 Structure of the Consent Object

Figure 5.6 shows how the validated information is combined to construct the consent object.

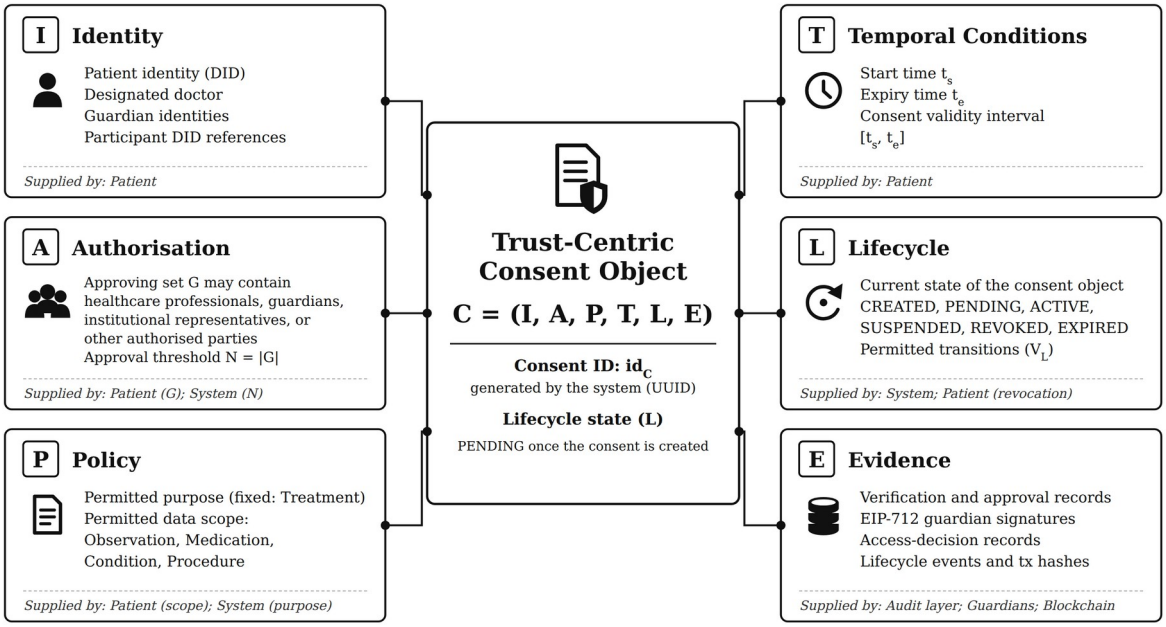


Figure 5.6: Six-component consent model C = (I, A, P, T, L, E).

Unlike the consent objects reported in the literature, this object carries the guardian set and threshold together with the purpose, data scope, validity interval, and lifecycle state.

5.5.3 Trust-Aware Consent Creation Procedure and Algorithm

The Trust-Aware Consent Creation procedure checks the participants, the purpose, the data scope and the validity interval, in that order, and only then stores the consent and logs its creation. The procedure consists of the following steps.

Step 1 – Validate Patient Identity. The system first validates the patient's identity Ip. In the model, Ip is checked through the identity and credential verification presented in Chapter 6; in the prototype, the patient DID is selected from the loaded record set and is declared rather than authenticated. If Ip cannot be successfully validated, the procedure terminates immediately and returns REJECT.
Step 2 – Validate the Authorised Doctor. The system validates the identity and authorisation status of the designated doctor D. The doctor DID must be well formed and must hold a valid doctor credential, which in the prototype is issued by a demo issuer. An invalid or unauthorised doctor results in rejection of the consent request.
Step 3 – Validate Guardian or Proxy Participants. If a guardian or proxy is specified, the system validates the guardian set G. Each guardian is checked to determine whether the individual or entity possesses the required authority to participate in consent-related decisions. In the prototype, at least one guardian must be listed, and each guardian DID must hold a valid guardian credential or the specification is rejected.
Step 4 – Validate the Intended Purpose. The intended data-use purpose Pu is validated to ensure that the purpose is explicitly specified and conforms to the supported healthcare-use policies. The purpose is

subsequently bound to the consent object so that authorisation is not interpreted as unrestricted access to the patient's healthcare information.
Step 5 – Validate the Data Scope. The requested data scope SC is validated to determine precisely which healthcare resources may be accessed under the consent. The scope is a set of resource categories; in the prototype these are the modules Observation, Medication, Condition and Procedure. Following the principle of least privilege, a consent names the data it covers rather than the patient's whole record, and the scope must not be empty.
Step 6 – Validate the Consent Validity Interval. The system validates the temporal validity interval T = (ts, te), where ts represents the consent start time and te represents the expiration time. The system verifies that the interval is logically valid by requiring te > ts. If this condition is not satisfied, the consent request is rejected.
Step 7 – Define the Authorisation Requirement. After all input elements have been validated, the system defines the authorisation requirement A. The requirement A names who must approve and the threshold their approvals must reach; it is what VA(C) later evaluates. In the prototype, A is the guardian set G with N set to the number of guardians, so every listed guardian must sign.
Step 8 – Generate a Unique Consent Identifier. The system generates a unique consent identifier idC. Every later step refers to the consent by idC: each guardian signature is bound to it, and the creation event is logged against it.
Step 9 – Construct the Consent Object. The validated information is combined to construct the consent object C = (I, A, P, T, L, E), where I represents the verified identity information, A represents the authorisation requirements, P represents the consent purpose and associated permissions, T represents the validity interval, L represents the lifecycle state, and E represents the evidence associated with the consent decision.
Step 10 – Persist the Consent Object. The constructed consent object C is persisted in the consent-management infrastructure. In the prototype the store is the local SQLite database; the object is written there in the state PENDING_SIGNATURE, and the later authorisation checks read that stored copy.
Step 11 – Record Consent Creation Evidence. Finally, the system records evidence associated with the creation event, including the consent identifier, participating entities, relevant timestamps, and the resulting consent state. This evidence provides an auditable record that can be used to reconstruct and verify the consent lifecycle.
Step 12 – Return the Consent Result. If all validation and construction operations are completed successfully, the system returns the newly created consent object C. If any mandatory validation fails during the process, the system terminates the procedure and returns REJECT.
The procedure can be expressed as the following trust establishment sequence: $Ip \rightarrow D \rightarrow G \rightarrow Pu \rightarrow SC \rightarrow T \rightarrow A \rightarrow idC \rightarrow C \rightarrow E$.

A failure at any validation step returns REJECT, so no consent is stored until the identity, authority, purpose, scope, and temporal checks have all passed.

Algorithm 1 Trust-Aware Consent Creation

Require: Patient identity I_p, authorised doctor D, guardian set G, purpose P_u, data scope S_C, validity interval T

Ensure: Consent object C or rejection

Validate I_p
if I_p is invalid **then**
    return REJECT
end if
Validate authorised doctor D, guardian set G, purpose P_u and data scope S_C
if any of them is invalid **then**
    return REJECT
end if
Validate validity interval $T = (t_s, t_e)$
if $t_e \leq t_s$ **then**
    return REJECT
end if
Define authorisation requirement A
Generate consent identifier id_C
Construct C = (I, A, P, T, L, E)
Persist C
Record consent creation evidence
return C

Algorithm 1 formalises the creation process. The algorithm does not grant data access directly. Its output is the object C; whether a request is allowed is decided later, against that object. The consent is placed in the PENDING state (PENDING_SIGNATURE in the prototype), and nothing is anchored on-chain until all guardians approve. The algorithm also establishes the relationship between RO1 and the remaining objectives. RO2 uses the identity and authority information contained in I, RO3 evaluates the authorisation requirements contained in A, and RO4 evaluates the policy constraints contained in P.

5.6 Prototype Implementation

RO1 has been completed at the conceptual and prototype level, and the HALAH patient interface creates the consent object, assigns the identifier, and records the creation event in the audit layer. The main contribution of RO1 is that consent is represented as an evaluable authorisation object rather than as a binary permission. In the current prototype, the patient DID is selected from the loaded record set. The doctor DID and each guardian DID must be well formed and must hold a valid credential for its role, issued by a demo issuer, before the consent is stored, as qualified in Chapter 6. The validation evidence for consent creation, including the demonstration scenarios and the unit-level tests, is presented in Section 5.9.

5.6.1 Aims of Engineering the Prototype System

The main purpose of the prototype was to simulate the operation of the HALAH framework, in which patient consent is created as an evaluable

object and every access request is evaluated against it. In this chapter we use the prototype to check two things about the consent model of Section 5.5: that a patient can create the object C = (I, A, P, T, L, E) through an interface, and that the object carries every component that the later checks need. For validating the other objectives of this thesis, we have extended the same prototype (see Chapters 6 to 9).

As discussed in Chapter 2, one of the major challenges of consent management in healthcare is that a stored record of permission does not establish who may act on it, whether every required party agreed, or whether a request stays within the purpose and the data scope that the patient set. With our framework, the consent object carries these conditions, and the prototype makes the design testable: the automated tests call the same code as the interface and can challenge the checks enforced by the interface directly at the service level. This testing exposed the gaps addressed by the hardening of 1 October 2026. The prototype also produces evidence: every creation, approval, decision and revocation is written to the audit ledger, so the claims of this and the following chapters can be traced to recorded events rather than to screenshots alone.

We used the Ethereum test network Sepolia as the computing platform for the on-chain part of the prototype, and Solidity as its programming language (Ethereum Foundation 2026a). Sepolia was chosen because it runs the same virtual machine as the Ethereum main network, so the contract executes as it would on the main network, while its ether has no monetary value, which allowed the mint and revoke transactions to be repeated during development. The prototype is built to validate the design, not to run in production: its Controller is a single trusted server, as Section 5.3 states.

5.7 Datasets Used for Validation

The prototype uses synthetic patient records from the Synthea 1K sample (Walonoski et al. 2018), taken from synthea.mitre.org in its CSV export rather than its FHIR R4 export (HL7 International 2019). Synthea generates realistic but artificial patients, so no real patient data is involved. The sample is public and contains 1,000 simulated patient records, covering demographics, medical conditions, medications and procedures. The prototype loads a subset of patients together with their conditions, observations, medications and procedures, and these tables provide the healthcare data modules that a consent can authorise, such as Observation, Medication and Procedure. The records are used only to construct representative consent workflows and the healthcare data-access scenarios used to test the prototype. The authorisation decisions do not depend on the clinical content, and no medical content is written to the chain. The records are not changed. The only step is the selection of the subset loaded by the prototype.

Table 5.3 shows the Synthea tables loaded by the prototype, the number of records in the loaded subset, and the consent module that each table supplies. The patient DID of the prototype is derived from the Synthea

patient identifier, and the records of a patient are selected by that identifier when a consent authorises the module.

Table 5.3: Synthea tables loaded by the prototype and the consent module supplied by each

Synthea table	Records in the loaded subset	Record selected by	Consent module
patients.csv	113	Id (the source of the patient DID)	Patient identity in the patient interface
observations.csv	4,428	PATIENT	Observation
medications.csv	2,931	PATIENT	Medication
conditions.csv	4,748	PATIENT	Condition
procedures.csv	4,237	PATIENT	Procedure

5.8 Steps in the Working of the Prototype System for Consent Creation

In order to create a consent, the HALAH prototype operates at two levels: the patient interface and the service layer. In the patient interface, the consent specification is entered and checked. In the service layer, the consent object is constructed, stored and recorded in the audit ledger. The steps at both levels follow Algorithm 1 and are explained below in Section 5.8.1 and Section 5.8.2 respectively, and Figure 5.7 shows the patient interface after a consent has been created.

5.8.1 Steps in the patient interface

1. The patient selects the role Patient and their own DID, which the prototype loads from the record set.

2. The patient enters the DID of the authorised doctor and the DIDs of the guardians, separated by commas.

3. The patient selects the data modules from Observation, Medication, Condition and Procedure, and sets the start and expiry times.

4. The patient presses Initial Consent. The prototype checks that at least one guardian is listed, that the doctor DID holds a valid doctor credential, that each guardian DID holds a valid guardian credential, that no guardian appears twice, that the data scope is not empty and that the expiry time is later than the start time. If any check fails, an error message names the failed field and nothing is stored.

5.8.2 Steps in the service layer

5. If every check passes, the service layer assigns the consent identifier, stores the object in the state PENDING_SIGNATURE with N set to the

number of guardians, writes the CREATE_CONSENT event to the audit ledger, and returns the identifier to the interface, which shows it to the patient.

6. The new consent appears in the patient's list with 0 of N approvals, where it waits for the guardians, whose steps are presented in Chapter 7.

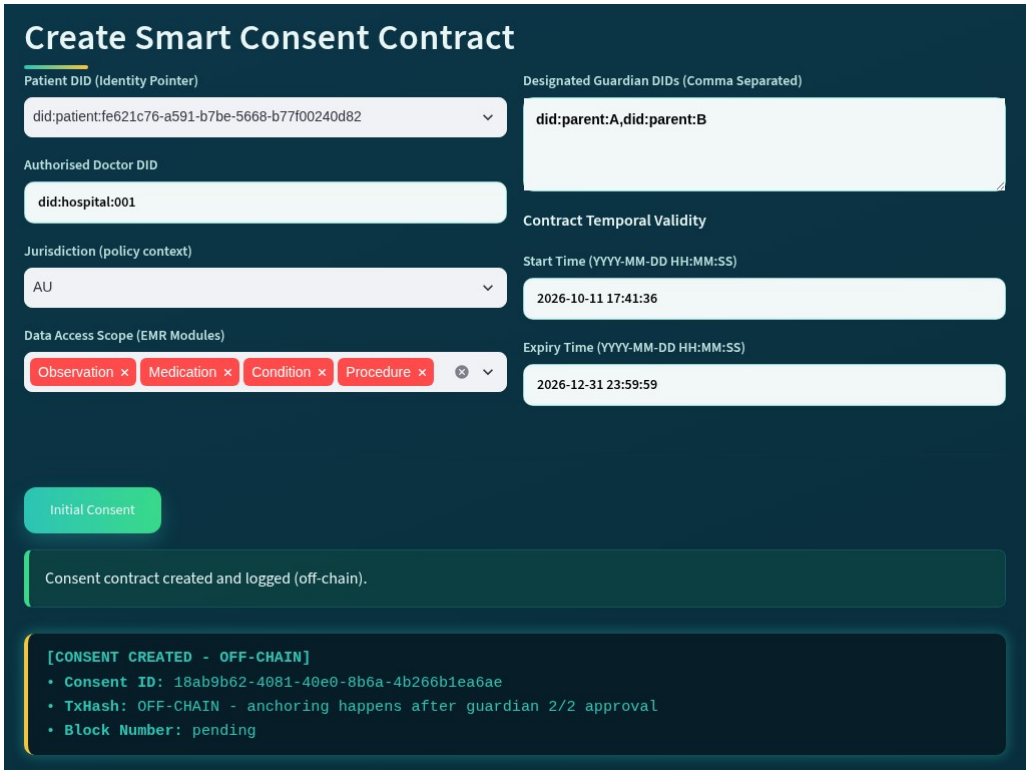


Figure 5.7: The patient interface of the HALAH prototype after a consent has been created.

5.9 Prototype Evaluation and Discussion

Objective one involves the representation of patient consent as an evaluable authorisation object. The output of RO1 is the conceptual model presented in Sections 5.2 to 5.5, rather than a set of measurements. The prototype results reported in this section show only that the model can be realised and behaves as specified, so they are reported as outcomes, created or rejected, rather than as numbers; the quantitative evaluation of the complete framework is given in Chapter 10. The results of the validation carried out for this objective are shown in Figure 5.8 and Table 5.4, and the security properties of the architecture are examined in Table 5.5. RO1 is validated in two ways. First, the creation of the consent object is checked functionally: a valid specification must produce a consent object with all six components, and an invalid specification must be rejected before anything is stored. Second, the architecture is evaluated against the security properties it is designed to provide. The validation of the predicates that are evaluated against the consent object, VI, VA, VP, VT and VL, belongs to the chapters of RO2 to RO5, and the evaluation of the complete decision belongs to Chapter 10.

5.9.1 Validation of Consent Creation

Live scenarios. In the demonstration of 24 September 2026, Scenario 1 created the consent f050c29a for a patient with two guardians, did:parent:A and did:parent:B, the authorised doctor did:hospital:001, the four data modules, the purpose Treatment and a validity window ending on 31 December 2026. The audit ledger holds the CREATE_CONSENT event for this consent at 09:50:49 with the result SUCCESS, followed by a second consent, 04da25f5, created at 10:55:49 for another patient. Both consents then went through the guardian approvals and the access decisions reported in the later chapters, so in these two cases the later checks could be run on each created object; the evidence for each component separately is given in Table 5.4.

Rejected specifications. In the local run of 11 October 2026, three invalid specifications were submitted before the valid one. A doctor DID that does not hold a valid doctor credential was rejected, a guardian DID that does not hold a valid guardian credential was rejected, and a specification that named the same guardian twice was rejected with the message shown in Figure 5.8. Nothing was stored and nothing was logged for the three rejected submissions. The fourth submission, with two distinct guardians holding valid credentials, created the consent 18ab9b62 at 17:41:36, which was then used for the access decisions reported in Chapters 6 to 9.

Automated tests. Two tests cover consent creation. The test test_patient_form_checks_the_doctor_and_the_guardians drives the patient form itself: it submits a doctor DID without a valid doctor credential, a guardian without a credential and a duplicate guardian, and asserts that each is refused with the expected error message, then submits a valid specification and asserts that the consent is created and logged. The test test_consent_persists_across_reload stores a consent, adds an approval, reloads the store from disk and asserts that the stored approval is unchanged. Both tests pass in the suite of 46 reported in Chapter 10.

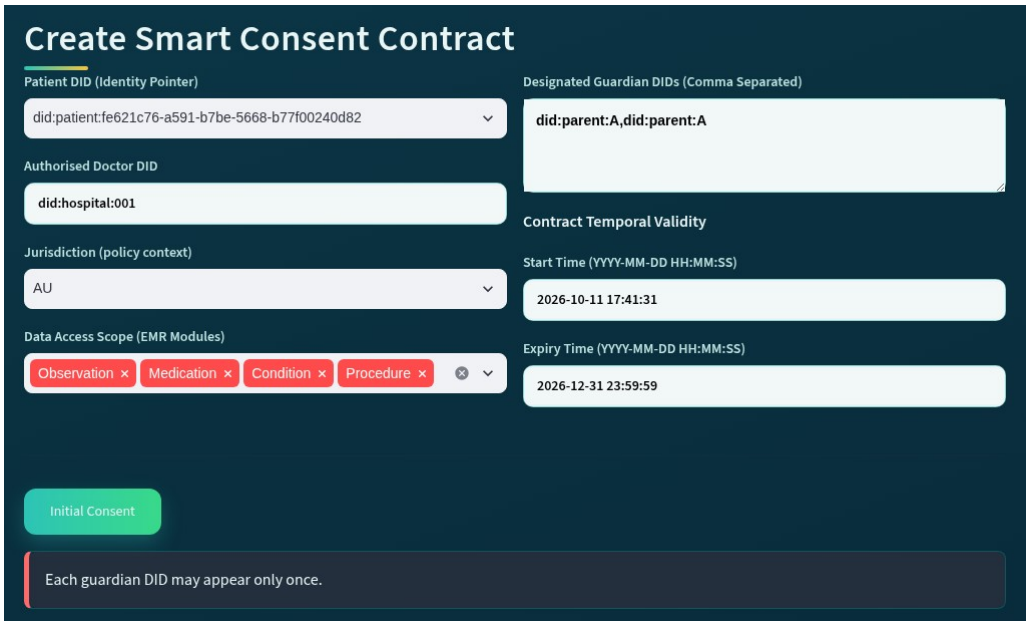


Figure 5.8: A specification that names the same guardian twice is rejected by Algorithm 1 before any consent is stored.

Table 5.4 brings these results together by component of the consent object. For each component of C it gives the check that Algorithm 1 applies at creation, the evidence from the live runs and the automated tests, and the result observed.

Table 5.4: Validation results of the prototype for consent creation

Component	**Check at creation (Algorithm 1)**	**Evidence**	**Result**
I Identity	Steps 1 to 3: the patient identity, the authorised doctor DID holding a valid doctor credential, and each guardian DID holding a valid guardian credential.	Run of 11 October 2026: a doctor DID without a valid doctor credential and a guardian DID without a valid guardian credential were rejected. Test test_patient_form_checks_the_doctor_and_the_guardians.	Identifiers without a valid credential rejected before anything is stored; valid identifiers holding valid credentials accepted.
A Authorisation	Steps 3 and 7: the guardian set G, and the threshold N derived from it as |G|; no guardian may appear twice.	A specification naming the same guardian twice was rejected (Figure 5.8). Scenario 1 of 24 September 2026 created consent f050c29a with the two guardians did:parent:A and did:parent:B.	Duplicate guardian rejected; consent created with N set to the number of guardians.
P Policy	Steps 4 and 5: the purpose and the data scope; the data scope must not be empty.	Scenario 1 recorded the four data modules and the purpose Treatment; the purpose vocabulary check is exercised by the automated tests.	Empty scope rejected; purpose and scope recorded in the consent object.
T Time	Step 6: the validity interval T = (ts, te); the expiry time must be later than the start time.	Scenario 1 recorded a validity window ending on 31 December 2026. The prototype rejects an expiry time that is equal to or earlier than the start time (Section 5.8).	Validity interval recorded; the interval is enforced at every request by VT (Chapter 9).
L Lifecycle	Steps 8 to 10: the consent identifier is generated, the object is constructed and persisted, and its state becomes PENDING_SIGNATURE.	Consents f050c29a, 04da25f5 and 18ab9b62 were created in PENDING_SIGNATURE. Test test_consent_persists_across_reload: the stored approval is unchanged after a reload from disk.	Created in PENDING_SIGNATURE; no access is possible until the state is ACTIVE (Chapter 9).
E Evidence	Step 11: the CREATE_CONSENT event is written to the audit ledger with the consent identifier and timestamp.	Ledger events at 09:50:49 (f050c29a), 10:55:49 (04da25f5) and 17:41:36 (18ab9b62), each with the result SUCCESS. Nothing was logged for the three rejected submissions.	Every creation is logged; a rejected specification stores and logs nothing.

5.9.2 Evaluation of the Architecture against the Security Properties

At this stage the architecture is designed to give the following security properties.

First, the architecture provides identity-dependent participation. In the prototype, the doctor DID and each guardian DID must be well formed and hold a valid role credential from a demo issuer, although the requester's DID is declared in the interface rather than authenticated (Section 5.2). Second, the architecture provides authority separation. Possession of a valid identity does not automatically result in permission to access patient data. Identity verification, once proof of DID control is added, provides evidence concerning who an entity is, whereas authority verification determines whether that entity may perform a particular role within the authorisation process. Third, the architecture provides collective authorisation integrity. Sensitive healthcare data access can be configured to require multiple independently verified participants. By design, VA(C) holds only once N signed approvals are present, so where more than one approval is required, no participant can reach the threshold alone. Fourth, the architecture provides policy-bounded access. VP(C, R) is designed to refuse a request that goes beyond the purpose or data scope in the consent object, even when the requester is verified and has enough approvals. Finally, the architecture provides decision accountability. The audit ledger records each decision, allowed or denied, with the result of every check, so an auditor can see why it was taken. These properties, together with the separation of interaction and enforcement established in Section 5.3, are summarised in Table 5.5, in which collective authorisation integrity and decision accountability appear as multi-party authorisation integrity and authorisation evidence respectively.

Table 5.5: Security properties supported by the proposed MVC-based consent architecture

Security property	**Architectural mechanism**	**Security significance**
Identity-dependent participation	DID and credential verification before a participant can contribute to the authorisation process	Designed to prevent unverified entities from being treated as legitimate participants.
Authority separation	Independent evaluation of identity and role or authority evidence	Designed to prevent successful authentication from being interpreted as unrestricted authorisation.
Multi-party authorisation integrity	Threshold evaluation over independently verified approval participants	Designed to prevent a single participant from unilaterally activating sensitive healthcare data access when collective approval is required.
Policy-bounded access	Evaluation of purpose and requested data scope against consent policy	Designed to prevent otherwise valid identities and approvals from being used to obtain data beyond the authorised scope.
Authorisation evidence	Recording of verification results, approval evidence, policy evaluation, and final decisions	Designed to provide traceability and support accountability, auditing, and future verification.
Separation of interaction and enforcement	The View submits requests while the Controller independently determines the security decision	Designed to prevent user-interface actions from being interpreted directly as trusted authorisation outcomes.

Each property in Table 5.5 is tested in a later chapter. Identity-dependent participation and authority separation are tested in Chapter 6 through the credential checks and the forged-credential attack. Multi-party authorisation integrity is tested in Chapter 7 through the outside-signer, replayed-signature and duplicate-approval attacks. Policy-bounded access is tested in Chapter 8 through the over-scope and empty-scope attacks. Authorisation evidence is checked in every chapter through the audit ledger and in Chapter 10 through the recorded decisions. The hardening of 1 October 2026 implemented the separation of interaction and enforcement: three attacks that the interface already blocked were also accepted by the service functions until the checks were moved into the Controller, as reported in Chapter 10. The architecture therefore does not claim security from MVC itself; it claims it from the placement of every trust-sensitive check inside the Controller, and the tests of the following chapters are the evidence for that claim.

5.9.3 Discussion

The consent object offers three advantages at creation: completeness, rejection before storage, and evidence. Completeness means that a created consent states every condition that the later checks need; the evidence for those conditions, such as credentials and signatures, is still checked when each request is evaluated. Rejection before storage means that an invalid specification leaves no record behind, so the store never holds a consent that cannot be evaluated. Evidence means that every creation is written to the audit ledger with its identifier and its time, so the later decisions can be traced back to the consent they were taken on.

On the other hand, the trade-off is rigidity. A consent cannot be changed after it has been created, so a change of doctor, guardian or data scope requires a new consent, and the credentials of the doctor and of every guardian must have been issued before the consent can be created (see Chapter 6). In the current interface, the purpose is also fixed to Treatment. We accepted this rigidity for now, since every stored consent is complete and every creation is recorded, which matters more at this stage. Consent modification is left to the next stage (Chapter 9).

5.10 Conclusion

This chapter described the research architecture on which the framework is built. We explained the three layers of the architecture, where trust is evaluated and where the boundaries between the layers lie, and how the architecture maps to the six research objectives. In addition, we proposed a model to represent patient consent as the evaluable object C = (I, A, P, T, L, E), created through Algorithm 1 only when every field is valid, assigned a unique identifier, persisted in the state PENDING_SIGNATURE and recorded in the audit ledger. This addressed the first research objective of this thesis.

We also described the prototype used for validating the solution to research objective 1. This was done using a public synthetic dataset. Three consents were created with all six components, three invalid specifications were rejected with nothing stored or logged, and the creation tests pass in the suite of 46 reported in Chapter 10 (see Table 5.4); Table 5.5 sets the architecture against the six security properties it is designed to provide. The design also has a limit: a consent cannot be changed once it has been created, so a user of HALAH must create a new one when its terms change; consent modification is left to the next stage (Chapter 9).

The next chapter will explain how to verify who the participants are and what authority they hold, which is the solution for research objective 2.